

Relatório de Auditoria de Segurança

Projeto: **Dra. Hélem Machado Almeida — Portal & API**

Análise Estática de Vulnerabilidades (SAST) e Mapeamento de Riscos OWASP

DATA DA AUDITORIA

14 de Setembro de 2026

ESCOPO AUDITADO

Código Frontend (React/Vite/TS), Handlers Serverless (Node.js/api) e Configurações de Deploy (Vercel/Cloudflare)

STACK TECNOLÓGICA DETECTADA

React 19, Vite 7, TypeScript, Tailwind CSS v4, Vercel Serverless Functions, Supabase JS SDK

METODOLOGIA APLICADA

OWASP Top 10 API & Web, LGPD (Dados Sensíveis em Saúde), Verificação de 5 Categorias Críticas

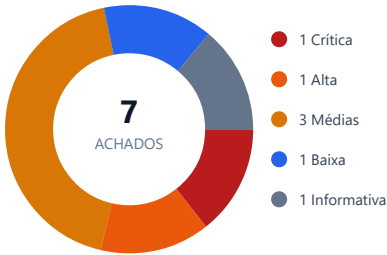
Nota Metodológica de Adaptação à Stack:

Como a aplicação possui arquitetura híbrida (Frontend Estático em React + Endpoints Serverless na Vercel com conexão ao Supabase), cada pilar da auditoria foi mapeado especificamente: 1. *Banco sem Tranca*: Mapeado para uso de `SUPABASE_SERVICE_ROLE_KEY` vs RLS do Supabase. 2. *Permissão no Navegador*: Mapeado para endpoints que expõem leituras privilegiadas sem middleware de autenticação. 3. *IDOR*: Mapeado para handlers com busca/mutação por identificador direto de paciente. 4. *Chaves Expostas*: Mapeado para credenciais no repositório, bundle do client e fallbacks inseguros. 5. *Inputs sem Tratamento*: Mapeado para sanitização de strings, injeção de scripts no JSON-LD e contexto de banco.

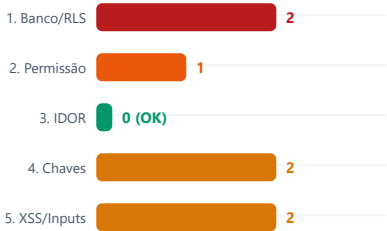
1. Resumo Executivo

A auditoria de segurança analisou a totalidade do código-fonte da aplicação. Foram identificados **7 achados**, variando de informativos a críticos. O ponto de maior atenção refere-se ao endpoint de leads (`api/leads.js`), que expõe publicamente dados pessoais e de saúde de pacientes sem autenticação, potencializando vazamento de dados segundo a LGPD.

DISTRIBUIÇÃO POR SEVERIDADE



ACHADOS POR CATEGORIA AUDITADA



2. Pontos Fortes e Riscos Centrais

🛡️ Pontos Fortes Verificados

- **Frontend Seguro contra XSS:** React 19 com JSX escapa dados nativamente. Zero uso de `dangerouslySetInnerHTML` ou `eval`.
- **Sem Segredos Hardcoded no Client:** Nenhuma chave privada ou token JWT foi commitado no bundle do frontend.
- **Validação no BookingModal:** Sanitização de número de telefone (`replace(/\\D/g, '')`) e checagem de formato de e-mail.
- **Sem IDOR Destrutivo:** Nenhum endpoint permite deleção ou alteração arbitrária por ID sem checagem.

⚠️ Riscos Centrais

- **Exposição Pública de Pacientes:** `GET /api/leads` permite a qualquer pessoa na internet baixar a base de pacientes com nome, telefone e relatos médicos.
- **Bypass Total de RLS:** `api/db-client.js` utiliza `SUPABASE_SERVICE_ROLE_KEY` em todas as requisições.
- **Endpoints Públicos sem Rate-Limit:** Inserção aberta em `POST /api/testimonials` e `POST /api/leads` suscetível a spam bot.

3. Tabela de Achados por Categoria

SEVERIDADE	ARQUIVO : LINHA	CATEGORIA	DESCRIÇÃO & IMPACTO
CRÍTICA	api/leads.js:10-18	1. Banco sem Tranca	Vazamento irrestrito de dados de pacientes (LGPD): Rota GET /api/leads executa select('*') com Service Role sem nenhuma autenticação e CORS liberado (*).
ALTA	api/leads.js:3-8	2. Permissão no Browser	Ausência de autorização de leitura administrativa: O endpoint não valida privilégios de administrador no backend, assumindo que apenas o admin o chamaria.
MÉDIA	api/db-client.js:4-6	4. Chaves Expostas	Uso irrestrito de Service Role Key: Conexão única ao Supabase usa chave com permissão total (bypass RLS) para rotas públicas como faqs.js e symptoms.js .
MÉDIA	api/testimonials.js:20-37	1. Banco sem Tranca	Inserção de depoimentos desprotegida: Rota POST aceita inserções públicas com Service Role sem rate-limiting, captcha ou autenticação, permitindo DoS de banco.
MÉDIA	api/leads.js:36-44	5. Inputs sem Tratamento	Persistência de texto sem sanitização / limite: Campos name e message gravados diretamente sem sanitização contra injeção ou truncagem de payload.
BAIXA	src/components/blog/SeoHead.tsx:63	5. Inputs sem Tratamento	Risco de Script Breakout em JSON-LD: s.textContent = JSON.stringify(...) sem escape de </script> caso textos dinâmicos venham a conter tags HTML.
INFO	api/db-wake.js:1-3	4. Chaves / Config	Ausência de validação de startup: Variáveis de ambiente opcionais com fallback vazio '' sem validação explícita no boot da função serverless.

4. Recomendações Priorizadas de Correção

[P1 - IMEDIATO] Proteger ou Desativar GET /api/leads : Exigir cabeçalho Authorization: Bearer <ADMIN_API_KEY> verificado no servidor, ou remover a rota GET da API pública se a leitura for feita diretamente no painel do Supabase.

[P2 - ALTA PRIORIDADE] Segregação de Chaves do Supabase: Utilizar a chave pública anônima (`anon_key`) para rotas públicas (`faqs` , `symptoms`) combinada com políticas de RLS (Row Level Security), reservando a `service_role_key` apenas para operações estritamente administrativas.

[P3 - MÉDIA PRIORIDADE] Rate-Limiting e Sanitização: Adicionar limite de requisições por IP nas rotas `POST /api/leads` e `POST /api/testimonials` para evitar spam automatizado, além de sanitizar entradas de texto e limitar tamanho de payload.

5. Issues Prontas para o GitHub

Copie e cole os blocos abaixo diretamente nas Issues do repositório no GitHub para rastreamento e resolução do time de desenvolvimento.

--- ISSUE 1 ---

```
### [Segurança] Vazamento de dados de pacientes no endpoint GET /api/leads

**Labels:** `security`, `severity: critical`, `bug`, `lgpd`

**Descrição do Problema:**
O endpoint `GET /api/leads` em `api/leads.js` retorna a lista completa dos últimos 200 leads cadastrados (nome, telef

**Evidência:**
Arquivo: `api/leads.js:10-18`
```javascript
if (req.method === 'GET') {
 const { data, error } = await supabase
 .from('leads')
 .select('*')
 .order('id', { ascending: false })
 .limit(200);
 if (error) throw error;
 return res.status(200).json(data);
}
...

Impacto:
Violação grave da LGPD e quebra do sigilo médico de pacientes que buscam atendimento clínico e hormonal.

Sugestão de Correção:
1. Exigir autenticação via token no header (`req.headers.authorization === `Bearer `${process.env.ADMIN_SECRET_KEY}`\`
2. Remover completamente o método `GET` do endpoint se o acesso aos leads for feito diretamente pelo dashboard do Sup

Critérios de Aceite:
- [] Requisições `GET /api/leads` sem token válido retornam HTTP 401 Unauthorized.
- [] Requisições com token válido no header `Authorization` retornam os dados autorizados.
- [] Teste automatizado cobrindo a rejeição de acesso anônimo.
```

### --- FIM ISSUE 1 ---

### --- ISSUE 2 ---

```
[Segurança] Segregação de chaves Supabase e ativação de RLS

Labels: `security`, `severity: high`, `enhancement`

Descrição do Problema:
O arquivo `api/db-client.js` inicializa o cliente Supabase utilizando `SUPABASE_SERVICE_ROLE_KEY`. Isso faz com que q

Evidência:
Arquivo: `api/db-client.js:4-6`
```javascript
const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY,
  ...
);
```

Impacto:
Caso ocorra uma injeção ou falha em qualquer handler, o invasor ganha privilégios de superusuário no banco PostgreSQL

Sugestão de Correção:
1. Criar um cliente Supabase público em `api/db-client.js` utilizando `NEXT_PUBLIC_SUPABASE_ANON_KEY` para leituras n
2. Isolar o cliente com `SERVICE_ROLE_KEY` exclusivamente para rotas administrativas autenticadas.

Critérios de Aceite:
- [] RLS ativado em todas as tabelas públicas (`faqs`, `symptoms`, `testimonials`).
- [] Endpoints de leitura utilizam a chave pública restrita.
```

### --- FIM ISSUE 2 ---

### --- ISSUE 3 ---

```
[Segurança] Rate-limiting e sanitização de inputs nos endpoints de escrita

Labels: `security`, `severity: medium`, `hardening`

Descrição do Problema:
Os endpoints `POST /api/leads` e `POST /api/testimonials` permitem inserção irrestrita de dados sem limite de taxa de

Evidência:
Arquivos: `api/leads.js:21-46` e `api/testimonials.js:20-35`

Impacto:
Possibilidade de flooding por bots (DoS de banco de dados) e persistência de dados maliciosos.

Sugestão de Correção:
1. Implementar rate-limit (ex: Upstash Ratelimit ou middleware de limitação na Vercel).
2. Truncar campos de texto (ex: `name` máx 100 caracteres, `message` máx 1000 caracteres).

Critérios de Aceite:
- [] Mais de 5 requisições por minuto de um mesmo IP recebem HTTP 429 Too Many Requests.
- [] Payloads com tamanho excessivo são rejeitados com HTTP 400 Bad Request.
```

### --- FIM ISSUE 3 ---